GLOSARIO

A - D

ACID: Propiedades de las transacciones en bases de datos: Atomicidad, Consistencia, Aislamiento, Durabilidad.

Activo digital: es cualquier elemento o recurso que existe en formato electrónico y tiene un valor económico, comercial o de uso.

Actuator: Módulo que expone endpoints para monitorizar la aplicación: salud, métricas, info, logs.

Application Context: El contenedor de IoC de Spring que centraliza, inicializa y configura todos los componentes de la aplicación.

application.properties / application.yml: Archivos de configuración centralizados donde se definen puertos, credenciales de bases de datos y variables del entorno.

Backend: Parte lógica y servidor de una aplicación que procesa los datos y gestiona la base de datos.

Backup: (o copia de seguridad) es el proceso de duplicar y almacenar datos informáticos en un soporte secundario con el objetivo de poder recuperarlos en caso de pérdida, corrupción, sabotaje o fallo del sistema original.

Beans: Objetos de Java que son creados, gestionados y ensamblados por el contenedor de Spring IoC.

Controller Layer / Service Layer / Repository Layer: Arquitectura por capas típica en Spring Boot:

Controller: recibe peticiones
Service: lógica de negocio
Repository: acceso a datos

DevTools: Herramientas para desarrollo: reinicio automático, LiveReload, mejoras de productividad.

E - K

Endpoint: es una URL específica asociada a un método en una clase controladora (@RestController o @Controller) que recibe peticiones HTTP de un cliente y devuelve una respuesta.

Entity (Entidad): Clase Java que representa una tabla en la base de datos, marcada habitualmente con @Entity.

Framework: Entorno de trabajo es una estructura conceptual y tecnológica de asistencia definida, normalmente, con artefactos o módulos concretos de software, que puede servir de base para la organización y desarrollo de software. Típicamente, puede incluir soporte de programas, bibliotecas, y un lenguaje interpretado, entre otras herramientas, para así ayudar a desarrollar y unir los diferentes componentes de un proyecto.

Frontend: Interfaz gráfica y visual con la que interactúa el usuario directamente en su dispositivo

Idempotencia: es una propiedad en matemáticas, lógica e informática que permite aplicar una acción varias veces y obtener el mismo resultado que si se hiciera una sola vez.
GET, PUT y DELETE son idempotentes (repetir la petición deja el recurso en el mismo estado).

Inversión de Control (IoC): Principio por el cual el contenedor de Spring toma el control del flujo del programa y gestiona el ciclo de vida de los objetos.

Inyección de Dependencias (DI): Patrón de diseño donde el framework proporciona automáticamente los objetos que una clase necesita para funcionar, en lugar de crearlos manualmente.

JPA / Hibernate: Tecnologías para persistencia de datos:

Se integran mediante spring-boot-starter-data-jpa
JPA: especificación
Hibernate: implementación más usada

L - O

Maven / Gradle: Herramientas de construcción que gestionan dependencias, compilación y empaquetado.

Maven: declarativo, basado en XML.
Gradle: programable, rápido, usa Groovy/Kotlin DSL.

Modelo-vista-controlador (MVC): es un patrón de arquitectura de software, que separa los datos y principalmente lo que es la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones

P - S

POJO (Plain Old Java Object): Objeto Java normal sin conocimiento de Spring. Los Beans pueden interactuar con POJOs.

Producto mínimo viable (MVP): versión básica de un nuevo producto o servicio que incluye solo las características esenciales para resolver el problema principal y satisfacer a los primeros clientes.

Repository (Interfaz): Abstracción de Spring Data que proporciona métodos automáticos (como save, findAll, delete) para interactuar con la base de datos sin escribir SQL.

Spinner: en CSS es una animación gráfica (un indicador de carga o loader) que gira de forma continua para informar al usuario que la página web o una aplicación está procesando información, cargando datos o esperando una respuesta.

Spring Boot: Framework de Java que simplifica el desarrollo de aplicaciones web y microservicios eliminando la mayor parte de la configuración manual.

Spring Boot Actuator: Módulo que añade funcionalidades para monitorear y gestionar la aplicación en producción (métricas, salud del sistema, etc.).

Starters: Dependencias preconfiguradas (ej. spring-boot-starter-web) que agrupan todas las librerías necesarias para una tarea específica.

T - Z

Toggle: interruptor de visibilidad en un formulario de login que sirve principalmente para alternar el campo de la contraseña entre texto oculto (type="password") y texto legible (type="text"), mejorando la experiencia del usuario al escribir.

UML: (por sus siglas en inglés, Unified Modeling Language, o Lenguaje Unificado de Modelado) es un lenguaje gráfico estándar para visualizar, especificar, construir y documentar los componentes de un sistema de software y modelos de negocio

Anotaciones Principales de Spring Bott (Annotations)

- **@SpringBootApplication:** Anotación de inicio que combina tres funciones básicas: configuración automática, escaneo de componentes y definición de la clase de configuración.
- @**Autowired:** Marca un punto donde Spring debe inyectar automáticamente una dependencia (un Bean) en una clase.
- @**Component:** Anotación genérica para indicar que una clase es un componente gestionado por Spring.
- @**Configuration:** Indica que una clase define métodos productores de Beans usando la anotación @Bean.

Arquitectura en Capas (Stereotypes)

- **@RestController:** Combina @Controller y @ResponseBody. Indica que la clase manejará peticiones HTTP y devolverá los datos directamente en formato JSON o XML.
- **@Service:** Marca clases que contienen la lógica de negocio principal de la aplicación.
- @**Repository:** Indica que la clase interactúa con la base de datos (capa de acceso a datos) y traduce las excepciones de almacenamiento.

Desarrollo Web y APIs (Spring MVC)

- **@RequestMapping:** Enruta peticiones HTTP a métodos específicos del controlador basándose en la URL.
- **@GetMapping / @PostMapping:** Atajos específicos para manejar peticiones HTTP GET (lectura) y POST (creación).
- **@PathVariable:** Extrae variables directamente de la URL de la petición (ej. /usuarios/{id}).
- **@RequestBody:** Convierte automáticamente el cuerpo de la petición HTTP (normalmente JSON) en un objeto Java.

https://albertprofe.dev/learning/learning-vocab-spring-boot.html

https://www.linkedin.com/pulse/conceptos-springboot-juan-sebasti%C3%A1n-fern%C3%A1ndez-montoya-spvve/

https://www.devtools.pt/es/dev/cheatsheets/spring-boot